Go for Senior .NET Engineers

Phase 3: Concurrency — Go's Killer Feature

9 Lessons: Goroutines, Channels, select, sync, context, Patterns

Current as of Go 1.25.6 (January 2026) — includes WaitGroup.Go, container-aware GOMAXPROCS

Lesson 25: Goroutines — Not Threads, Not Tasks

Goroutines are Go's concurrency primitive. They are lightweight green threads managed by the Go runtime, not OS threads. Each goroutine starts with roughly 2-4 KB of stack (which grows dynamically), compared to 1-8 MB for an OS thread. You can easily run millions of goroutines on a single machine.

25.1 — Starting a Goroutine

C# / .NET

```
// C# — Task-based
Task.Run(() => {
    Console.WriteLine("async work");
});

await Task.WhenAll(
    Task.Run(() => DoWork1()),
    Task.Run(() => DoWork2())
);
```

Go

```
// Go — goroutine
go func() {
    fmt.Println("async work")
}()

// No built-in "await all" —
// use sync.WaitGroup or channels
// (covered in next lessons)
```

25.2 — Goroutine vs OS Thread vs C# Task

Feature	Goroutine vs Thread vs Task
Initial memory	~4 KB (goroutine) vs ~1 MB (OS thread) vs ~1 KB (Task struct)
Scheduling	Go runtime M:N scheduler (user-space)
Creation cost	~0.3 µs (goroutine) vs ~100 µs (OS thread)
Max practical count	Millions (goroutine) vs Thousands (threads)
Preemption	Cooperative + async preemption (Go 1.14+)
Stack growth	Dynamic (2KB → 1GB) vs Fixed (OS thread)

25.3 — The go Keyword

```
// Any function call can become a goroutine
go processOrder(order)
go logger.Flush()
go func(msg string) {
    fmt.Println(msg)
}("hello from goroutine")

// CRITICAL: main() does NOT wait for goroutines
func main() {
```

```
    go fmt.Println("this may never print")
    // main exits immediately → program ends
}
```

WARNING: If main() returns, ALL goroutines are killed immediately. There is no graceful shutdown by default. You must use WaitGroup, channels, or context to coordinate goroutine lifetimes.

25.4 — GOMAXPROCS and Scheduling

```
import "runtime"

// Go 1.25+: GOMAXPROCS auto-detects container CPU limits
// In Kubernetes with cpu limit=4, GOMAXPROCS defaults to 4
fmt.Println(runtime.GOMAXPROCS(0))  // query current value

// Goroutines are multiplexed onto OS threads by the scheduler
// M:N scheduling: M goroutines on N OS threads
// The scheduler uses work-stealing for load balancing
```

TIP: Go 1.25 made GOMAXPROCS container-aware. In Kubernetes, it now respects CPU limits from cgroups automatically. Previously you needed third-party libs like uber-go/automaxprocs.

Lesson 26: Channels — Typed, Buffered, Directional

Channels are Go's primary mechanism for goroutine communication. The Go proverb: 'Don't communicate by sharing memory; share memory by communicating.' Channels replace shared-state concurrency (locks) with message-passing concurrency (CSP model).

26.1 — Channel Basics

```
// Create an unbuffered channel (synchronous)
ch := make(chan string)

// Send a value (blocks until someone receives)
go func() {
    ch <- "hello"  // send
}()

// Receive a value (blocks until someone sends)
msg := <-ch
fmt.Println(msg)  // "hello"
```

26.2 — Buffered vs Unbuffered

```
// Unbuffered: sender blocks until receiver is ready
ch1 := make(chan int)      // capacity 0 (synchronous)

// Buffered: sender blocks only when buffer is full
ch2 := make(chan int, 10)  // capacity 10

// Buffered channel as a semaphore (limit concurrency)
sem := make(chan struct{}, 5)  // max 5 concurrent ops
for _, task := range tasks {
    sem <- struct{}{}  // acquire slot (blocks if 5 active)
    go func(t Task) {
        defer func() { <-sem }()  // release slot
        process(t)
    }(task)
}
```

26.3 — Directional Channels

```
// Channel direction enforced at compile time
func producer(out chan<- int) {  // send-only
    for i := 0; i < 10; i++ {
        out <- i
    }
    close(out)
}

func consumer(in <-chan int) {  // receive-only
    for val := range in {
```

```
        fmt.Println(val)
    }
}

func main() {
    ch := make(chan int, 5)  // bidirectional
    go producer(ch)         // narrows to send-only
    consumer(ch)            // narrows to receive-only
}
```

26.4 — Closing Channels and range

```
// Only the SENDER should close a channel
// Closing signals "no more values"
close(ch)

// range loops until channel is closed
for val := range ch {
    process(val)
}

// Check if channel is closed with comma-ok
val, ok := <-ch
if !ok {
    fmt.Println("channel closed")
}
```

WARNING: Sending on a closed channel panics. Closing an already-closed channel panics. Receiving from a closed channel returns the zero value immediately. Only close channels from the sender side.

Lesson 27: select Statement — Multiplexing Channels

The select statement lets a goroutine wait on multiple channel operations simultaneously. It is the equivalent of a switch for channels and is fundamental to all advanced concurrency patterns.

27.1 — Basic select

```
select {
case msg := <-msgChan:
    fmt.Println("received:", msg)
case err := <-errChan:
    fmt.Println("error:", err)
case <-time.After(5 * time.Second):
    fmt.Println("timeout!")
}

// If multiple cases are ready, one is chosen at random
// If no case is ready, select blocks until one is
```

27.2 — Non-Blocking Operations with default

```
// Non-blocking send
select {
case ch <- value:
    fmt.Println("sent")
default:
    fmt.Println("channel full, dropping")
}

// Non-blocking receive
select {
case val := <-ch:
    fmt.Println("got:", val)
default:
    fmt.Println("nothing available")
}
```

27.3 — Infinite Loop with select (Event Loop Pattern)

```
func eventLoop(ctx context.Context, events <-chan Event, ticks <-chan time.Time)
{
    for {
        select {
        case evt := <-events:
            handleEvent(evt)
        case <-ticks:
            flushMetrics()
        case <-ctx.Done():
            fmt.Println("shutting down:", ctx.Err())
```

```
            return
        }
    }
}
```

27.4 — C# select Equivalent

C# / .NET

```
// C# — Task.WhenAny
var completed = await Task.WhenAny(
    GetMessageAsync(),
    GetErrorAsync(),
    Task.Delay(TimeSpan
        .FromSeconds(5))
);

if (completed == msgTask)
    Console.WriteLine(msgTask.Result);
```

Go

```
// Go — select
select {
case msg := <-msgChan:
    fmt.Println(msg)
case err := <-errChan:
    fmt.Println(err)
case <-time.After(5 * time.Second):
    fmt.Println("timeout")
}
```

TIP: select is how you build timeouts, heartbeats, cancellation listeners, and rate limiters in Go. It replaces C#'s Task.WhenAny, CancellationToken polling, and event-driven patterns.

Lesson 28: sync Package — Mutex, RWMutex, WaitGroup, Once

While channels are Go's preferred synchronization mechanism, the sync package provides traditional locking primitives for cases where shared state is simpler than message passing.

28.1 — sync.WaitGroup (Await All Goroutines)

```
var wg sync.WaitGroup

for _, url := range urls {
    wg.Add(1)
    go func(u string) {
        defer wg.Done()
        resp, err := http.Get(u)
        if err != nil {
            log.Println(err)
            return
        }
        defer resp.Body.Close()
        fmt.Printf("%s: %d\n", u, resp.StatusCode)
    }(url)
}

wg.Wait()  // blocks until counter reaches 0
fmt.Println("all requests complete")
```

28.2 — sync.Mutex and sync.RWMutex

C# / .NET

```
// C# — lock keyword
private readonly object _lock = new();
private int _counter;

lock (_lock) {
    _counter++;
}

// ReaderWriterLockSlim for R/W
var rwLock = new ReaderWriterLockSlim();
```

Go

```
// Go — sync.Mutex
var mu sync.Mutex
var counter int

mu.Lock()
counter++
mu.Unlock()

// sync.RWMutex for R/W
var rw sync.RWMutex
```

```
// Idiomatic: embed mutex in the struct
type SafeCounter struct {
    mu    sync.RWMutex
    count map[string]int
}

func (c *SafeCounter) Inc(key string) {
    c.mu.Lock()
    defer c.mu.Unlock()
```

```
    c.count[key]++
}

func (c *SafeCounter) Get(key string) int {
    c.mu.RLock()
    defer c.mu.RUnlock()
    return c.count[key]
}
```

28.3 — sync.Once (Thread-Safe Singleton)

```
// Execute a function exactly once, no matter how many goroutines call it
var once sync.Once
var db *sql.DB

func GetDB() *sql.DB {
    once.Do(func() {
        var err error
        db, err = sql.Open("postgres", connStr)
        if err != nil {
            log.Fatal(err)
        }
    })
    return db
}
```

TIP: Use channels for orchestration (passing data between goroutines). Use mutexes for protecting shared state (counters, caches, maps). Don't mix both approaches for the same data.

Lesson 29: context.Context — Cancellation, Timeouts, Values

context.Context is Go's answer to CancellationToken, request-scoped values, and timeout propagation — all in one type. It is the FIRST parameter of virtually every function in production Go code.

29.1 — C# CancellationToken vs Go context.Context

C# / .NET

```
// C# — CancellationToken
var cts = new CancellationTokenSource(
    TimeSpan.FromSeconds(5));

await DoWork(cts.Token);

async Task DoWork(
    CancellationToken ct) {
    while (!ct.IsCancellationRequested) {
        await ProcessAsync(ct);
    }
}
```

Go

```
// Go — context.Context
ctx, cancel := context.WithTimeout(
    context.Background(),
    5*time.Second)
defer cancel()

DoWork(ctx)

func DoWork(ctx context.Context) {
    for {
        select {
        case <-ctx.Done():
            return // cancelled or timed
out
        default:
            process(ctx)
        }
    }
}
```

29.2 — Creating Contexts

```
// Root context (no cancellation, no deadline)
ctx := context.Background()

// With cancellation
ctx, cancel := context.WithCancel(parentCtx)
defer cancel()  // ALWAYS defer cancel

// With timeout
ctx, cancel := context.WithTimeout(parentCtx, 30*time.Second)
defer cancel()

// With deadline
deadline := time.Now().Add(1 * time.Minute)
ctx, cancel := context.WithDeadline(parentCtx, deadline)
defer cancel()

// With value (request-scoped metadata)
ctx = context.WithValue(ctx, "requestID", "abc-123")
```

```
reqID := ctx.Value("requestID").(string)
```

29.3 — Propagation Through Call Stack

```
// Context flows through your entire call chain:
func HandleRequest(w http.ResponseWriter, r *http.Request) {
    ctx := r.Context()  // HTTP server provides context

    user, err := userService.GetUser(ctx, userID)
    // → calls repo.FindByID(ctx, userID)
    //   → calls db.QueryRowContext(ctx, query, id)
    //     → cancelled if client disconnects
}
```

29.4 — Context Best Practices

```
// 1. ALWAYS pass context as first parameter named ctx
func Process(ctx context.Context, data []byte) error

// 2. ALWAYS defer cancel()
ctx, cancel := context.WithTimeout(parent, 10*time.Second)
defer cancel()  // prevents goroutine leak

// 3. NEVER store context in a struct field
// BAD:  type Server struct { ctx context.Context }
// GOOD: pass ctx through method parameters

// 4. Use context.WithValue sparingly
// Only for request-scoped data (traceID, auth token)
// Not for passing function parameters
```

WARNING: Forgetting defer cancel() is the #1 cause of goroutine leaks in Go production code. Every WithCancel/WithTimeout/WithDeadline MUST have a matching defer cancel().

Lesson 30: Concurrency Patterns — Fan-In, Fan-Out, Worker Pool, Pipeline

These are the building blocks of every concurrent Go application. Understanding them is like understanding async/await patterns in C# — they show up everywhere in production code.

30.1 — Fan-Out (Distribute Work)

```
// Fan-out: one channel, multiple goroutines consuming from it
func fanOut(ctx context.Context, input <-chan Task, workers int) <-chan Result {
    results := make(chan Result, workers)
    var wg sync.WaitGroup

    for i := 0; i < workers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for task := range input {
                results <- process(ctx, task)
            }
        }()
    }

    go func() {
        wg.Wait()
        close(results)
    }()

    return results
}
```

30.2 — Fan-In (Merge Results)

```
// Fan-in: merge multiple channels into one
func fanIn(channels ...<-chan Result) <-chan Result {
    merged := make(chan Result)
    var wg sync.WaitGroup

    for _, ch := range channels {
        wg.Add(1)
        go func(c <-chan Result) {
            defer wg.Done()
            for val := range c {
                merged <- val
            }
        }(ch)
    }

    go func() {
        wg.Wait()
        close(merged)
```

```
    }()

    return merged
}
```

30.3 — Worker Pool (Most Common Pattern)

```
func WorkerPool(ctx context.Context, jobs []Job, workerCount int) []Result {
    jobCh := make(chan Job, len(jobs))
    resultCh := make(chan Result, len(jobs))

    // Start workers
    var wg sync.WaitGroup
    for i := 0; i < workerCount; i++ {
        wg.Add(1)
        go func(id int) {
            defer wg.Done()
            for job := range jobCh {
                select {
                case <-ctx.Done():
                    return
                case resultCh <- execute(job):
                }
            }
        }(i)
    }

    // Send jobs
    for _, j := range jobs {
        jobCh <- j
    }
    close(jobCh)

    // Wait and collect
    go func() { wg.Wait(); close(resultCh) }()

    var results []Result
    for r := range resultCh {
        results = append(results, r)
    }
    return results
}
```

30.4 — Pipeline (Staged Processing)

```
// Stage 1: Generate
func generate(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        for _, n := range nums {
            out <- n
        }
```

```
        close(out)
    }()
    return out
}

// Stage 2: Square
func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        for n := range in {
            out <- n * n
        }
        close(out)
    }()
    return out
}

// Stage 3: Print
func main() {
    // Pipeline: generate → square → print
    for val := range square(generate(2, 3, 4)) {
        fmt.Println(val)  // 4, 9, 16
    }
}
```

TIP: Pipelines in Go are like UNIX pipes. Each stage is a goroutine, connected by channels. Data flows one direction. This pattern is used extensively in data processing, ETL, and streaming systems.

Lesson 31: Race Conditions & the -race Detector

Data races occur when two or more goroutines access shared memory concurrently and at least one writes. Go ships with a built-in race detector that instruments your code at compile time.

31.1 — A Data Race Example

```
// BUG: data race on counter
var counter int

func main() {
    for i := 0; i < 1000; i++ {
        go func() {
            counter++  // unsynchronized read-modify-write
        }()
    }
    time.Sleep(time.Second)
    fmt.Println(counter)  // unpredictable result
}
```

31.2 — Using the Race Detector

```
# Build and run with race detection
go run -race main.go
go test -race ./...
go build -race -o myapp

# Output on race detection:
# WARNING: DATA RACE
# Write at 0x00c0000b4010 by goroutine 7:
#   main.main.func1()
#       /app/main.go:12 +0x38
# Previous read at 0x00c0000b4010 by goroutine 6:
#   main.main.func1()
#       /app/main.go:12 +0x30
```

31.3 — Fixing the Race

```
// Fix 1: Use sync.Mutex
var mu sync.Mutex
var counter int

mu.Lock()
counter++
mu.Unlock()

// Fix 2: Use sync/atomic (best for simple counters)
var counter atomic.Int64
counter.Add(1)
fmt.Println(counter.Load())
```

```
// Fix 3: Use a channel (idiomatic Go)
counterCh := make(chan int, 1)
counterCh <- 0
// increment: val := <-counterCh; counterCh <- val + 1
```

WARNING: ALWAYS run go test -race in CI. The race detector has ~10x performance overhead, so use it in testing, not production. It catches real bugs that are nearly impossible to find otherwise.

Lesson 32: sync/atomic — Low-Level Atomic Operations

The sync/atomic package provides lock-free atomic operations for simple shared state. Since Go 1.19, it includes typed wrappers (atomic.Int64, atomic.Bool, etc.) that are safer and more readable than the raw functions.

32.1 — Typed Atomics (Go 1.19+)

```
import "sync/atomic"

// Typed atomic values (preferred since Go 1.19)
var requestCount atomic.Int64
var isRunning   atomic.Bool
var config      atomic.Pointer[Config]

// Increment
requestCount.Add(1)

// Load
count := requestCount.Load()

// Store
isRunning.Store(true)

// Compare-And-Swap
swapped := isRunning.CompareAndSwap(true, false)

// Pointer swap (lock-free config reload)
newCfg := &Config{MaxConns: 200}
config.Store(newCfg)
current := config.Load()
```

32.2 — C# Interlocked vs Go atomic

C# / .NET

```
// C# — Interlocked
private long _count;

Interlocked.Increment(ref _count);
Interlocked.Add(ref _count, 5);
var val = Interlocked.Read(ref _count);

Interlocked.CompareExchange(
    ref _count, newVal, expected);
```

Go

```
// Go — sync/atomic
var count atomic.Int64

count.Add(1)
count.Add(5)
val := count.Load()

count.CompareAndSwap(
    expected, newVal)
```

32.3 — When to Use atomic vs Mutex

Use atomic When	Use Mutex When
Single counter or flag	Multiple fields updated together
Read-heavy, write-rare	Complex state transitions
Performance is critical	Code clarity matters more
Lock-free is required	You need to hold lock across operations

TIP: Atomic operations are 2-10x faster than mutexes for simple operations. But they only work for single values. For multi-field updates, always use a Mutex to avoid partial updates.

Lesson 33: errgroup — Structured Concurrency

The golang.org/x/sync/errgroup package provides structured concurrency: launch multiple goroutines, wait for all of them, and return the first error. It combines WaitGroup + error handling + context cancellation into one clean API.

33.1 — Basic errgroup Usage

```
import "golang.org/x/sync/errgroup"

func FetchAll(ctx context.Context, urls []string) ([]string, error) {
    g, ctx := errgroup.WithContext(ctx)
    results := make([]string, len(urls))

    for i, url := range urls {
        i, url := i, url  // capture loop variables
        g.Go(func() error {
            body, err := fetchURL(ctx, url)
            if err != nil {
                return fmt.Errorf("fetch %s: %w", url, err)
            }
            results[i] = body  // safe: each goroutine writes to unique index
            return nil
        })
    }

    if err := g.Wait(); err != nil {
        return nil, err  // returns FIRST error
    }
    return results, nil
}
```

33.2 — C# Task.WhenAll vs Go errgroup

C# / .NET

```
// C#
try {
    var tasks = urls.Select(
        url => FetchAsync(url, ct));
    var results = await Task
        .WhenAll(tasks);
    return results;
} catch (Exception ex) {
    // first exception thrown
    throw;
}
```

Go

```
// Go
g, ctx := errgroup.WithContext(ctx)

for i, url := range urls {
    i, url := i, url
    g.Go(func() error {
        results[i], err = fetch(ctx, url)
        return err
    })
}

err := g.Wait() // first error
```

33.3 — errgroup with Concurrency Limit

```
// Limit concurrent goroutines (Go 1.20+)
g, ctx := errgroup.WithContext(ctx)
g.SetLimit(10)  // max 10 concurrent goroutines

for _, task := range tasks {
    task := task
    g.Go(func() error {
        return processTask(ctx, task)
    })
}

if err := g.Wait(); err != nil {
    return fmt.Errorf("processing failed: %w", err)
}
```

33.4 — WaitGroup.Go (Go 1.25+)

Go 1.25 added a Go() method to sync.WaitGroup, simplifying the common Add(1)/go func()/defer Done() pattern. For error-free concurrent work, this replaces the old boilerplate.

```
// Before Go 1.25
var wg sync.WaitGroup
for _, item := range items {
    wg.Add(1)
    go func(it Item) {
        defer wg.Done()
        process(it)
    }(item)
}
wg.Wait()

// Go 1.25+ with WaitGroup.Go
var wg sync.WaitGroup
for _, item := range items {
    item := item
    wg.Go(func() {
        process(item)
    })
}
wg.Wait()
```

33.5 — Phase 3 Summary: Concurrency Concept Map

C# / .NET Concept	Go Equivalent
Task / Task<T>	goroutine (go keyword)
async / await	No equivalent — goroutines block naturally
Task.WhenAll	sync.WaitGroup or errgroup

`Task.WhenAny`	`select statement`
`CancellationToken`	`context.Context`
`CancellationTokenSource`	`context.WithCancel / WithTimeout`
`Channel<T>`	`chan T (built-in)`
`lock / Monitor`	`sync.Mutex`
`ReaderWriterLockSlim`	`sync.RWMutex`
`Interlocked`	`sync/atomic (atomic.Int64, etc.)`
`SemaphoreSlim`	`Buffered channel (chan struct{})`
`Lazy<T>`	`sync.Once`
`BlockingCollection<T>`	`Buffered channel with range`
`Parallel.ForEach`	`Worker pool pattern`
`Task.Run + error handling`	`errgroup.Group`

TIP: Go's concurrency model is fundamentally different from C#'s. There is no async/await coloring problem. Every function can block without 'infecting' its callers. Goroutines are cheap enough that blocking is fine — the scheduler handles the rest.